北京开源芯片研究院
BEIJING INSTITUTE OF OPEN SOURCE CHIP

中国科学院计算技术研究所
INSTITUTE OF COMPUTING TECHNOLOGY, CHINESE ACADEMY OF SCIENCES

UNITYCHIP
万众一芯

UCAgent: An End-to-End Agent for Block-Level Functional Verification

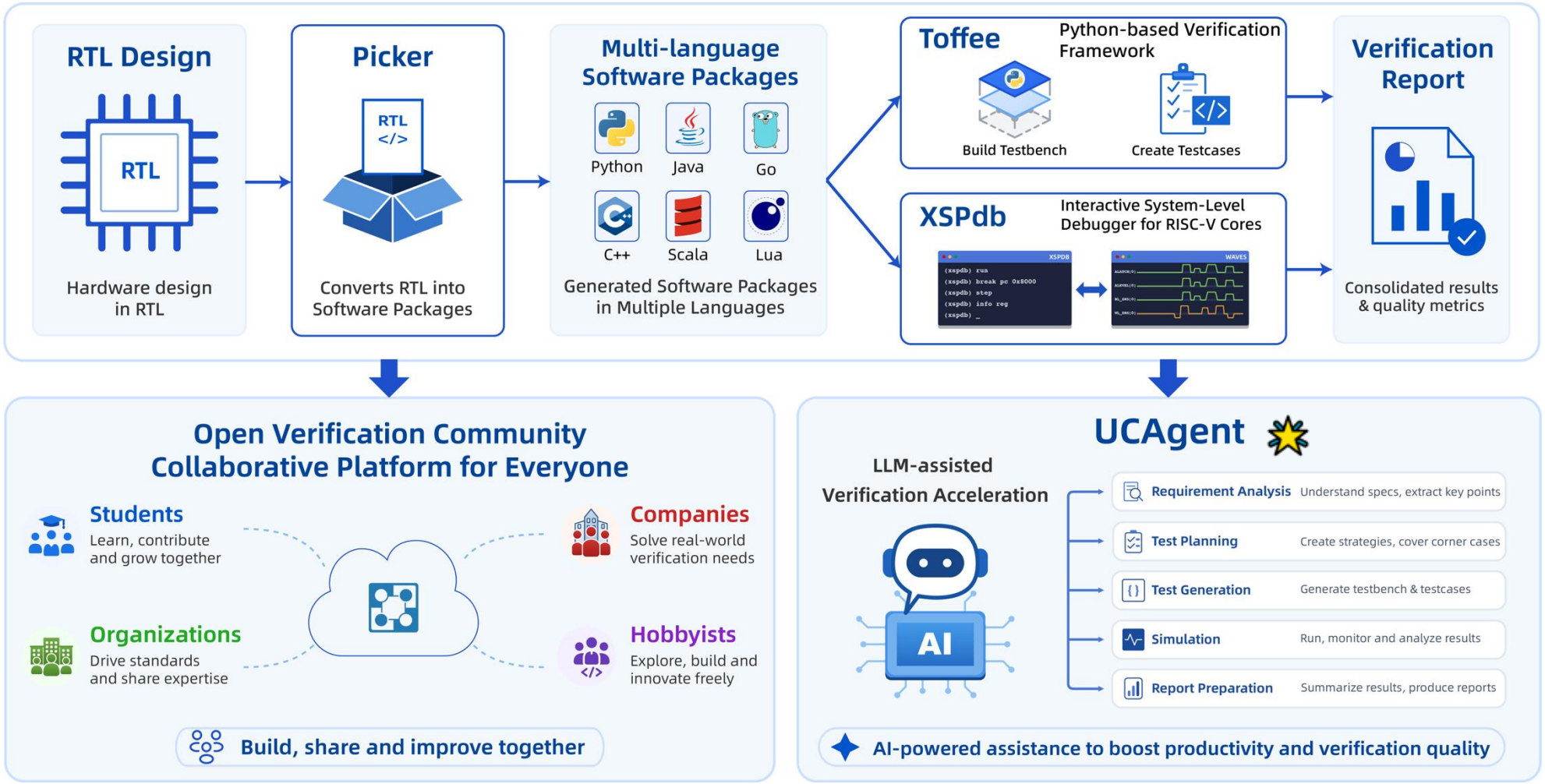
Junyue Wang

8th, June, 2026

Who We Are: UnityChip

- An open-source community of chip verification
- Dedicated to **exploring AI and software-driven** approaches for verification
  - A series of open-source tools for verification
  - Try to improves efficiency and lowers adoption barriers
  - **XiangShan** is the first projects using UnityChip

Our Works and Vision



Before start, let's download the necessary packages

A. Access Web UI Only

1. **Connect WiFi**
   - SSID: BCC_26
   - Password: Bologna26
2. **Visit Address**
   - http://172.29.16.123:8800

B. Try UCAgent (Local)

1. **Pull Images from Github**

```
docker pull \
        ghcr.io/xs-mlvp/workshop:latest
```

See Next Page for execution commands

https://github.com/XS-MLVP/tutorial-records

.../blob/master/workshop/README.md

Demo Video

北京开源芯片研究院 BEIJING INSTITUTE OF OPEN SOURCE CHIP

中国科学院计算技术研究所 INSTITUTE OF COMPUTING TECHNOLOGY, CHINESE ACADEMY OF SCIENCES

UNITYCHIP 万众一芯

UCAgent: An End-to-End Agent for Block-Level Functional Verification

Junyue Wang

8th, June, 2026

Contents

Verification Is the Critical Bottleneck

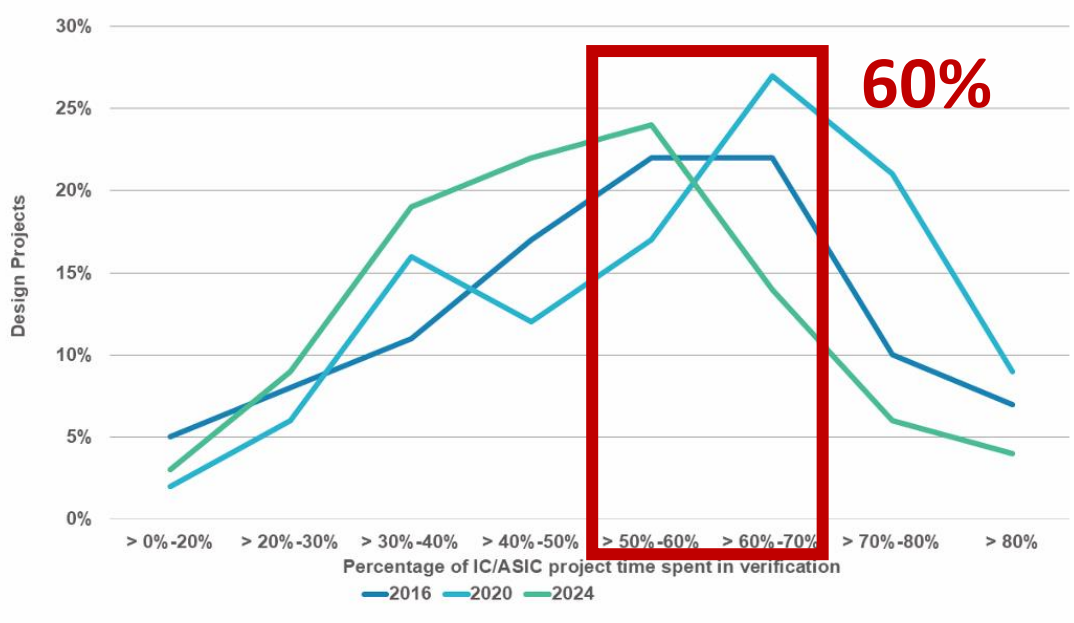


Figure 7. Percentage of IC/ASIC project time spent in verification.

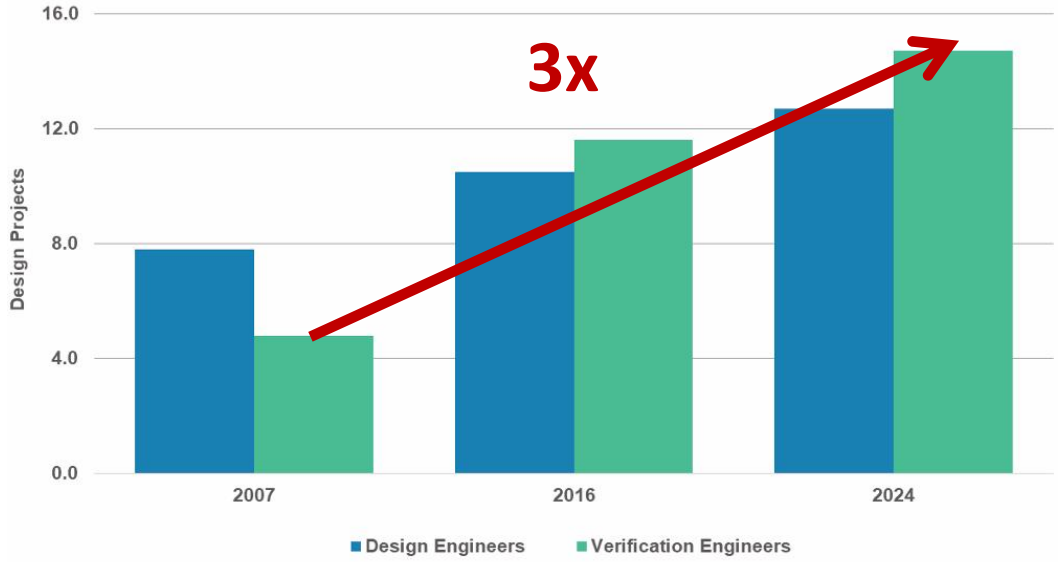


Figure 8. Mean peak number of IC/ASIC engineers.

- Spend **50–70% of total project time** on verification.
- Verification engineer demand **increasing by ~3x** from 2007 to 2024

Verification has become a major bottleneck in IC/ASIC development

*Source: SIEMENS, **2024 Wilson Research Group IC/ASIC functional verification trend report***
https://resources.sw.siemens.com/zh-TW/white-paper-2024-wilson-research-group-ic-asic-functional-verification-trend-report/

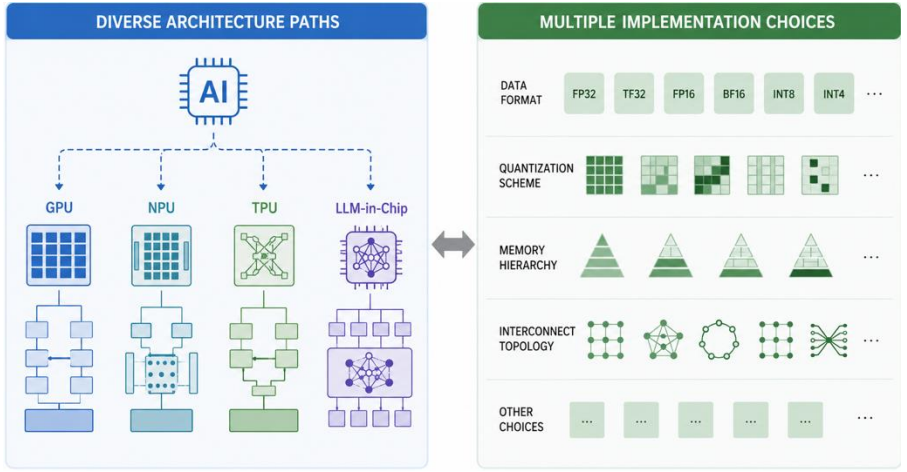
Rising Design Demand Makes Verification Harder

- Two trends are increasing design complexity
  - **Growing RISC-V Market Demand**

    Drives more customization designs
  - **AI Computing Demand**

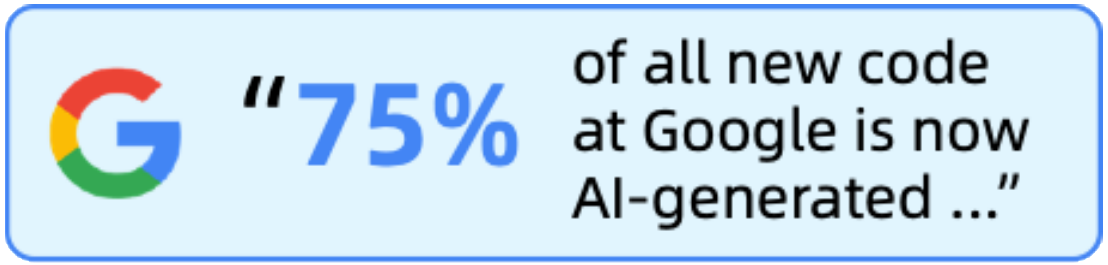
    Drives more AI-oriented designs

Growing design diversity creates more verification scenarios



Left Picture Source: The SHD Group, ***RISC-V Outperforming Expectations, RISC-V Summit North America 2025***
Right Picture Source: Generated by GPT-image-2, OpenAI

LLMs Work Well in Software



> ... The real long pole in that space is **design verification**. And so we're particularly looking at **how we can use AI to prove the designs work more quickly** ...
>
> — Bill Dally, NVIDIA GTC 2026

Can we bring LLMs to verification?

Left Sources:

- *Sundar Pichai, **Cloud Next '26: Momentum and innovation at Google scale, Google Cloud Blog, Apr 2026.** https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/cloud-next-2026-sundar-pichai*
- *Katherine Tangalakis-Lippert, **Anthropic CFO says AI now writes 90% of its code, changing white-collar jobs from execution to oversight, Business Insider, May 2026.** https://www.businessinsider.com/anthropic-cfo-white-collar-jobs-changed-execution-oversight-2026-5*

*Right Source: Bill Dally and Jeff Dean, **Advancing to AI's Next Frontier: Insights From Jeff Dean and Bill Dally, NVIDIA GTC 2026, Mar 2026.** https://www.nvidia.com/en-us/on-demand/session/gtc26-s82167/*

Verification Looks Similar to Software Testing

Similarities Between **Functional Verification** and **Software Testing**

Objective

Find defects in software / hard-ware under test

Process

Test planning
Testcase generation
Regression
Bug tracking

Metrics

Coverage
Pass/Fail results
Bug status

These similarities make LLM-based verification **possible**

Verification is Different from Software Testing

But they are **manageable**

Language

SystemVerilog

vs.

Python, Java, C++, ...

Both use the **object-oriented** programming paradigm

Execution Model

Cycle-level Concurrency

Signal Timing

...

Can be modeled as **FSMs driven by events and signal transitions**

Simulation Semantics

Clocks

Delta Cycles

...

These details can be wrapped by tools, like **Picker**

Contents

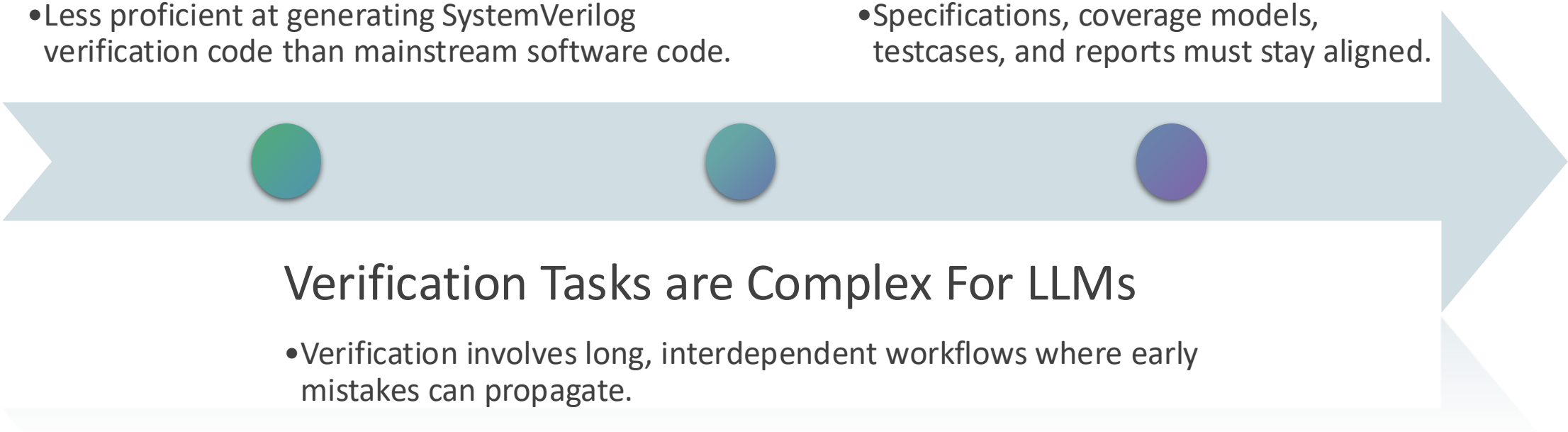
Key Challenges in LLM-Based Functional Verification

LLMs Struggle with Generating HDL Verification Code

- Less proficient at generating SystemVerilog verification code than mainstream software code.

Verification Consistency Is Hard to Maintain

- Specifications, coverage models, testcases, and reports must stay aligned.

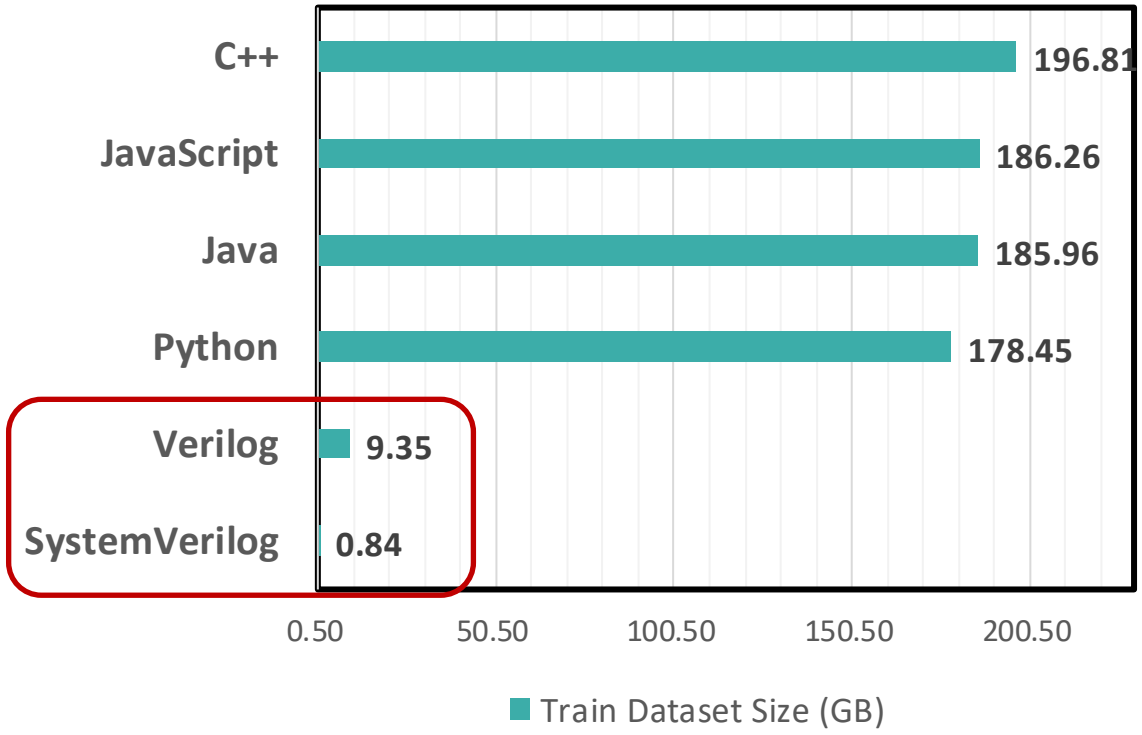


Verification Tasks are Complex For LLMs

- Verification involves long, interdependent workflows where early mistakes can propagate.

Challenge 1: LLMs Struggle with Generating HDL Verification Code

- The scarcity of data limits the reliability of generated verification code.



"**Design verification categories** - specificallly testbench stimulus and checker generation (**cid12-13**) and assertion generation (**cid14**) - **exhibit substantially lower pass rates …**"

Model	cid12	cid13	cid14
Claude 3.7 Sonnet	25.0%	6.0%	19.0%
" Thinking	24.0%	7.0%	23.0%
Claude 3.5 Haiku	16.0%	3.0%	11.0%
GPT 4.1	16.0%	10.0%	12.0%
GPT o1	15.0%	9.0%	5.0%
GPT o4-mini	13.0%	11.0%	10.0%
Llama 3.1 405B	21.0%	5.0%	13.0%

SystemVerilog Verification Tasks Reach **Only 3%–25%** Pass@1

Left Source: Lozhkov et al. ***StarCoder 2 and The Stack v2: The Next Generation.***
https://huggingface.co/datasets/bigcode/the-stack-v2/blob/main/language_stats.csv
Right Source: N. Pinckney et al. ***Comprehensive Verilog Design Problems: A Next-Generation Benchmark Dataset for Evaluating Large Language Models and Agents on RTL Design and Verification.***

Challenge 2: Verification Tasks are Complex for LLMs

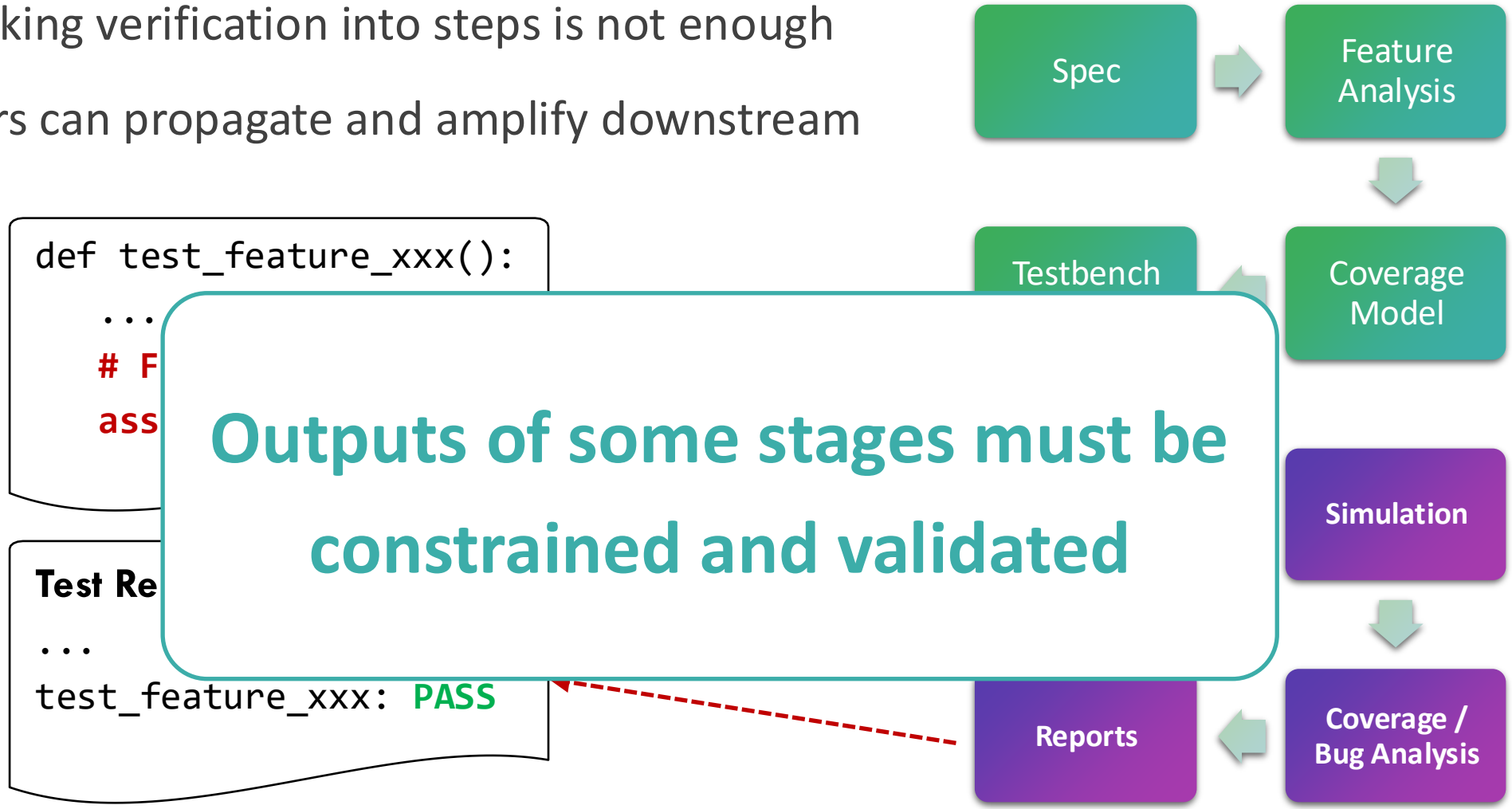
- Verification is **not a problem that a single prompt can solve**
- Requires long-text understanding and reasoning
- Use one-shot prompt, like:

We need to break the verification tasks into multiple steps

Picture Source: https://www.facebook.com/groups/cursedaiwtf/posts/1799163287358813/

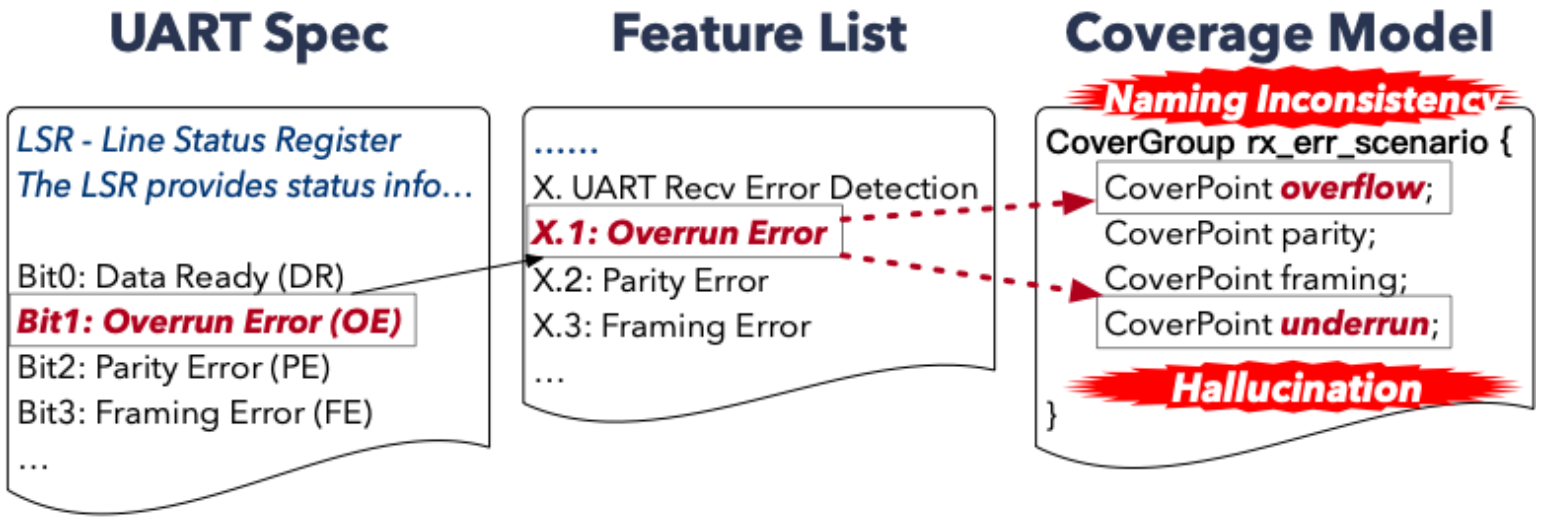
Challenge 2: Verification Tasks are Complex for LLMs

- Breaking verification into steps is not enough
- Errors can propagate and amplify downstream



Challenge 3: Maintaining Consistency Across the Full Flow Is Difficult

- A robust and reliable verification process requires strict **consistency**



Verification results are only trustworthy when

goals, implementations, and results remain aligned

Related Works: LLM-Assisted Verification

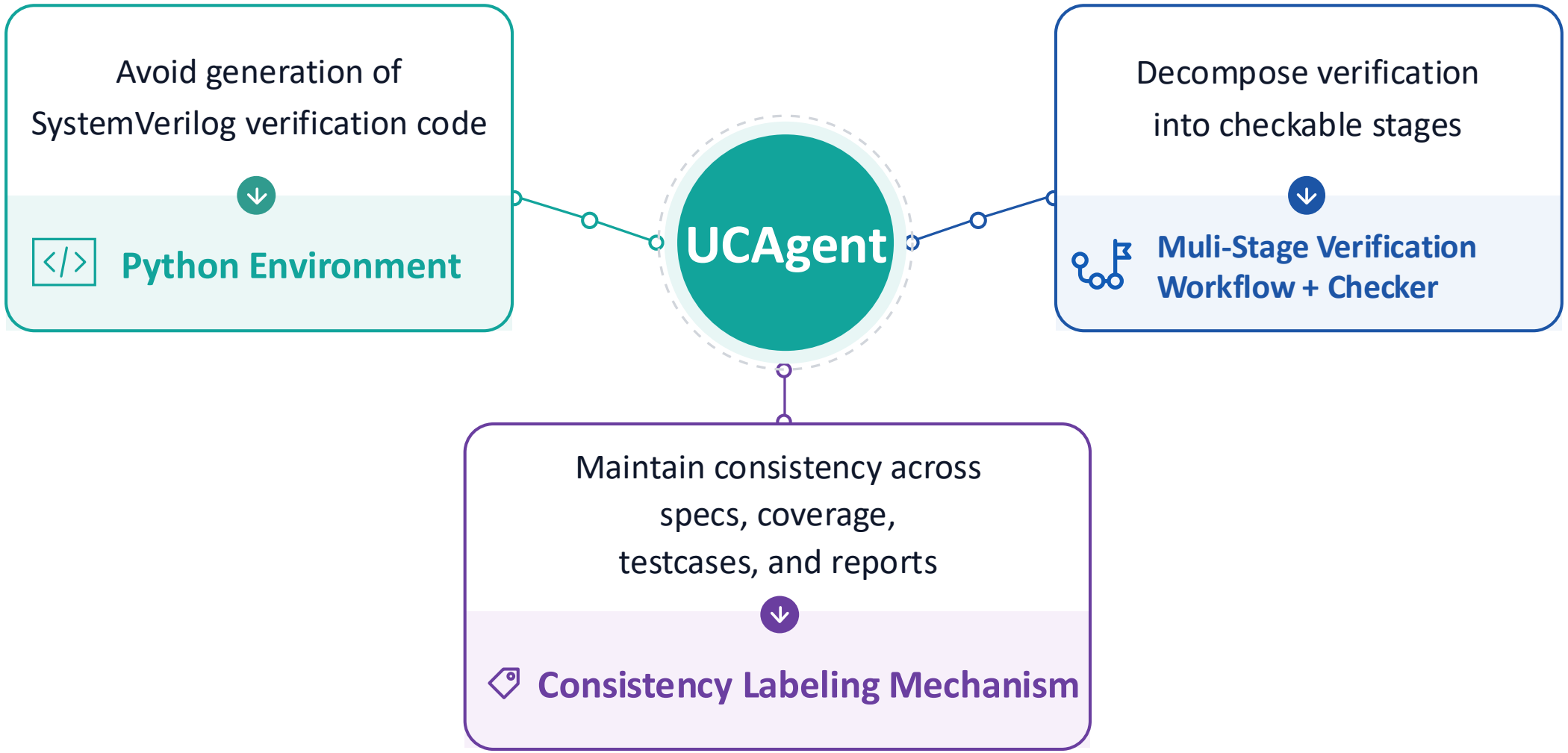
1. SV / UVM-based	2. Python-based	3. Commercial EDA Agent
UVM², MAVF	PRO-V	ChipStack
• Generates UVM-style testbenches and use coverage feedback • For UVM², **prompt-only constraints** are not enough. • Still need **manual intervention**	• Python generation helps produce runnable testbenches • Tis results **lack functional coverage feedback** • Consistency across spec, tests, and reports is not explicit	• Multiple scenario-specific agents support different verification tasks • **Not open-source** • Hard to reuse as an open, configurable workflow

UCAgent targets an open, checkable, and reliable verification workflow

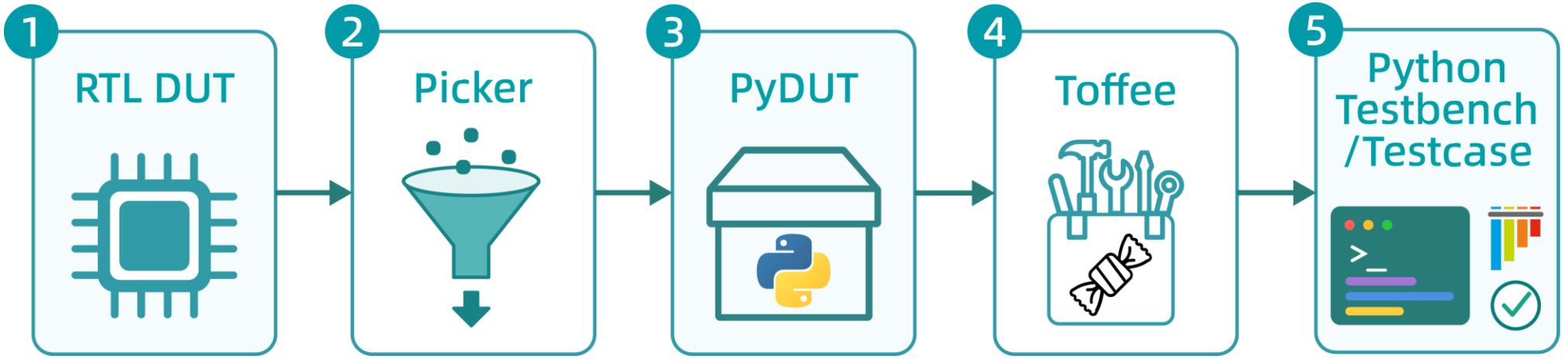
Sources:
Junhao Ye et al. **From Concept to Practice: an Automated LLM-aided UVM Machine for RTL Verification. arXiv:2504.19959.**
Wenbo Liu et al. **A Multi-Agent Generative AI Framework for IC Module-Level Verification Automation. arXiv:2507.21694.**
Yujie Zhao et al. **PRO-V: An Efficient Program Generation Multi-Agent System for Automatic RTL Verification. arXiv:2506.12200.**

Contents

Our Solution: UCAgent



Mechanism 1: Python Verification Environment



Write Python verification code instead of generating SV/UVM code

Benefits

- Uses a language which LLMs are good at
- Wraps low-level hardware interaction details
- Improves generation stability and iteration speed

What is Picker?

Converts RTL designs into software libraries

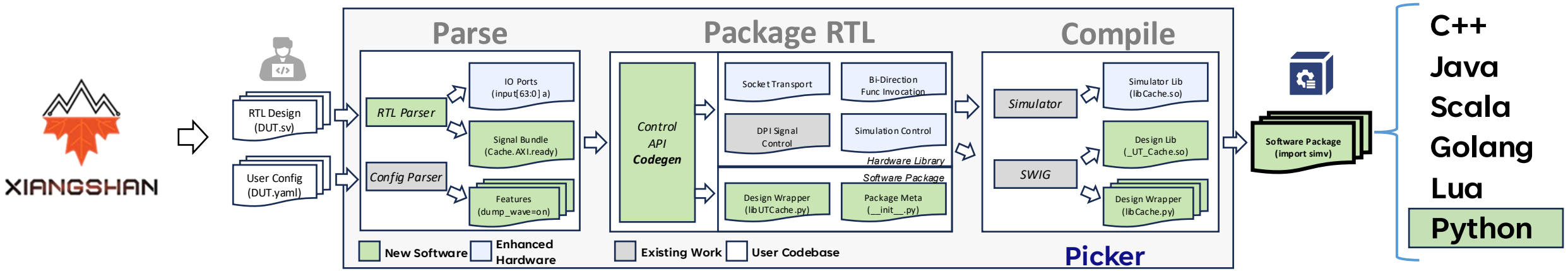
- Wraps low-level hardware interaction details
- Near-zero performance loss, on par with VCS / Verilator

github.com/XS-MLVP/picker

Accepted by ISCA '26

- Democratizing and Accelerating Hardware Verification with Software-Native Optimization

A typical workflow of Picker:




```
$picker export RAS.v --fs dep.f --lang python –tdir RAS
```

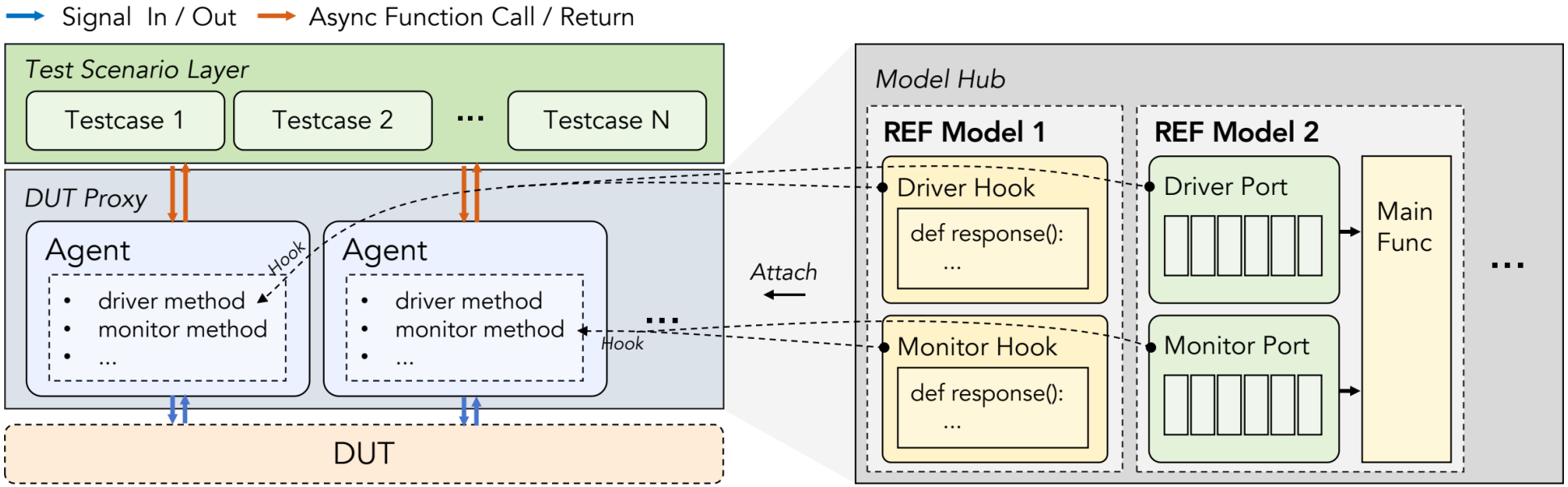
What is Toffee?

UVM-like Python Verification Framework

- Driver, Monitor, Agent, Env, ...
- Testcase Management
- Based on PyDUT from Picker

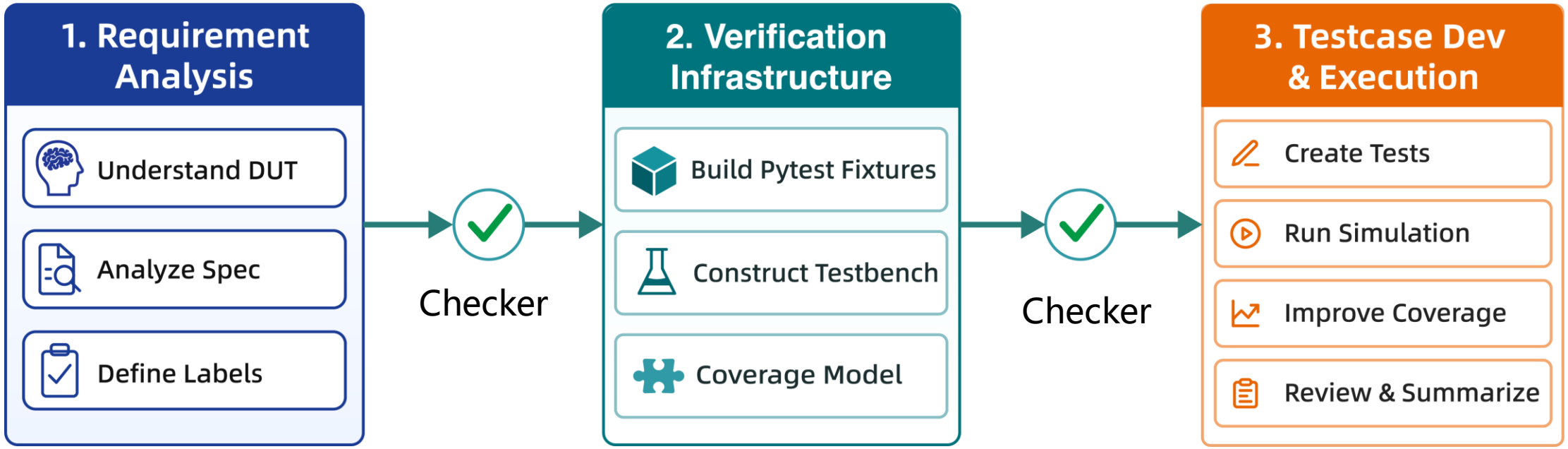
Integrates Pytest Ecosystem

github.com/XS-MLVP/toffee



Mechanism 2: Multi-Stage Workflow + Checker

Decompose end-to-end verification into **small, explicit, and checkable stages**



Mechanism 2: Multi-Stage Workflow + Checker

- Current workflow has **28** checkable stages
- And **25** different checkers

```
 0  1-requirement_analysis_and_planning-需求分析与验证规划 (0 fails, 02m 27s)
 1  2-dut_function_understanding-e203_exu_alu功能理解 (0 fails, 01m 11s)
 2  3.1-dut_function_grouping-DUT功能分组与层次划分 (0 fails, 03m 14s)
 3  3.2-function_point_definition-具体功能点识别与定义 (0 fails, 01m 27s)
 4  3.3-check_point_design-检测点设计与定义 (0 fails, 02m 51s)
 5  3-functional_specification_analysis-功能规格分析与测试点定义 (0 fails, 58s)
 6  4-static_bug_analysis-RTL源码静态Bug分析[13/13] (0 fails, 03m 26s)
 7  5.1-dut_creation_implementation-DUT创建函数实现 (0 fails, 46s)
 8  5.2-pytest_fixture_dut_implementation-dut fixture实现 (0 fails, 33s)
 9  5-dut_wrapper_implementation-DUT封装 (0 fails, 25s)
10  6.1-coverage_group_creation-功能覆盖组创建 (0 fails, 01m 01s)
11  6.2.1-implemente_function_checks_in_batch-分批功能点检查函数实现[102/102] (2
fails, 09m 17s)
```

```
13  6-coverage_model_implementation-功能覆盖率模型实现 (0 fails, 01m 50s)
14 *7.1-human_check_env_specification-人工检查env规格说明 (skipped)
15  7.2-bundle_wrapper_design-引脚封装设计 (0 fails, 02m 17s)
16  7.6-env_fixture_implementation-env fixture实现 (0 fails, 45s)
17  7.7-evaluate_env_fixture-env fixture实现测试与评估 (0 fails, 03m 28s)
18 *7.8-human_check_env_implementation-人工检查env实现质量 (skipped)
19  8-basic_api_implementation-基础API实现 (0 fails, 01m 49s)
20  11-basic_api_functional_test-基础API功能正确性测试 (6 fails, 05m 56s)
21  12-create_test_case_templates-创建测试用例模板[-/-|-] (0 fails, 05m 16s)
22  13.1-test_case_implementation_in_batch-分批测试用例实现与对应bug分析[-/-] (10
fails, 50m 14s)
23  13-comprehensive_verification_and_bug_analysis-全面验证与缺陷分析 (0 fails, 01m
23s)
24  14-static_bug_validation-静态分析Bug验证与动态关联 (0 fails, 04m 12s)
25  15-line_coverage_analysis_and_improvement-代码行覆盖率分析与提升(90.00) (skipped)
26  16-generate_random_test_cases-随机测试用例生成 (0 fails, 02m 02s)
27  17-verification_review_and_summary-验证审查与总结 (0 fails, 02m 02s)
```

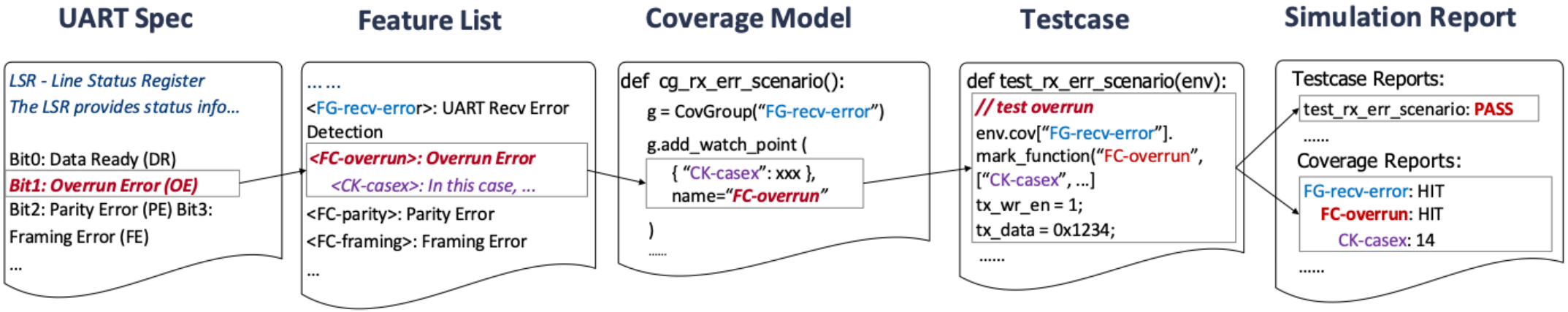
Mechanism 3: Verification Consistency Labeling Mechanism

Label Hierarchy

- FG: Function Group
- FC: Function Checkpoint
- CK: Checkpoint

Checker Validates

- Label completeness
- Cross-stage matching
- Consistency from goals to results



Make verification intent **explicit, structured, and machine-checkable**

Contents

Results Overview

Benchmark DUTs

48 runs from 12 DUTs × 4 LLMs

42 runs ≥95% functional coverage

33 runs ≥95% code coverage

Engineering Case Studies

Radix-4 SRT Integer Divider

Verification Acceleration

- 45–51 min vs ~3 days
- 100% functional coverage
- 99.69% line coverage

Vector Floating-Point Mixed Adder

Bug Discovery

- 3 confirmed bugs
- 5 hours runtime

Benchmark DUTs & LLMs

Control

Module	Line of Code
e203_ifu_litebpu	55
sdc_controller	1681
e203_lsu_ctrl	348
ICache-WaylookUp	3808

Datapath

Module	Line of Code
aes_key_expand_128	296
e203_exu_alu	1003
sd_crc_16	27
IntegerDivider	1458

Protocol & Buffering

Module	Line of Code
sd_controller_wb	478
sd_fifo_tx_filler	421
sd_tx_fifo	230
UART-16550	1433

Claude Sonnet 4.5 GPT-5.2 GLM-4.7 Minimax M2.1

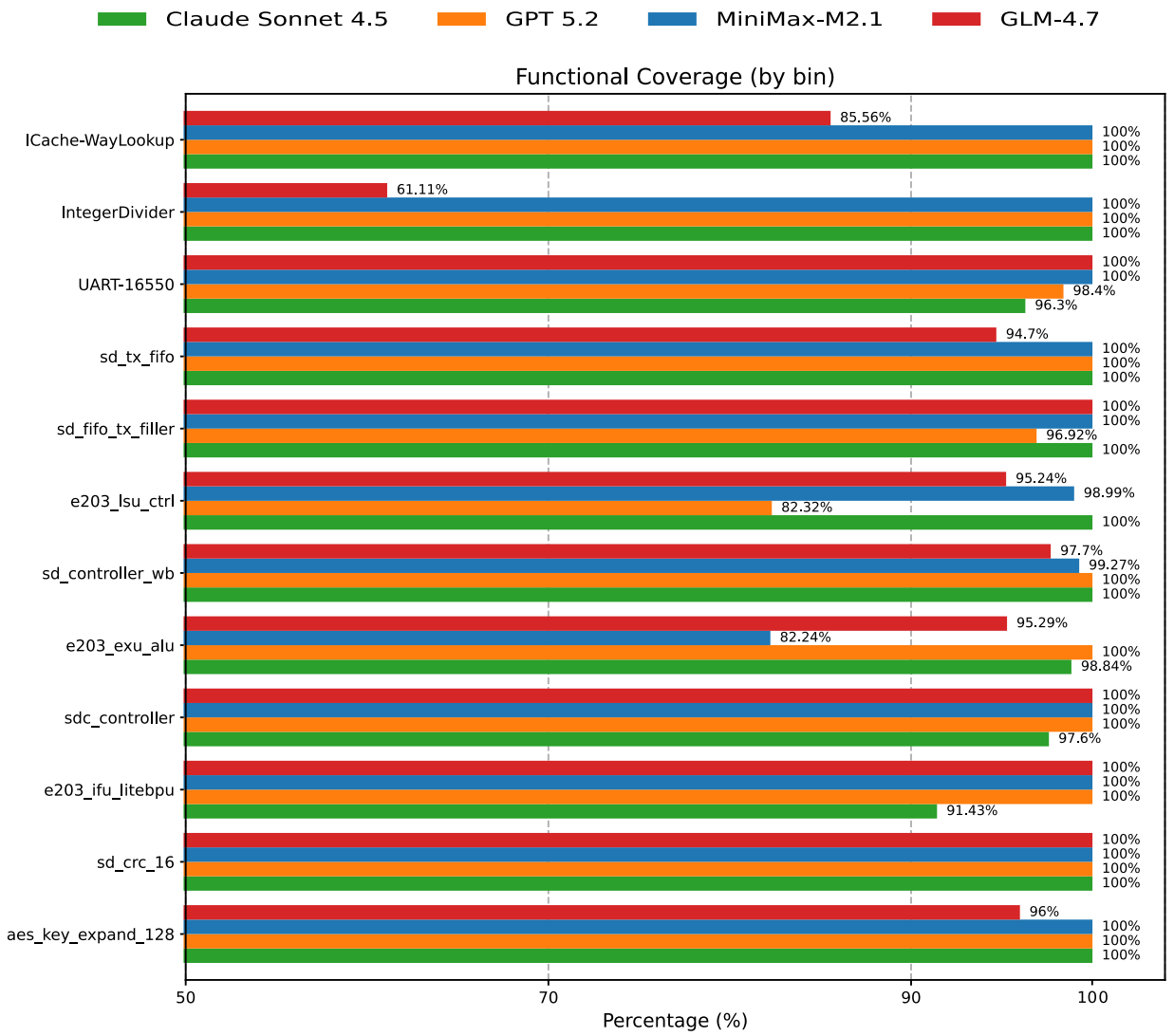
Benchmark Results: Functional Coverage

LLMs	Coverage ≥ 95%
Claude Sonnet 4.5	11/12
GPT 5.2	11/12
MiniMax-M2.1	11/12
GLM-4.7	9/12

Comparison Across LLMs

Module-Category	Coverage ≥ 95%
Control	13/16
Datapath	14/16
Protocol & Buffering	15/16

Comparison Across DUTs



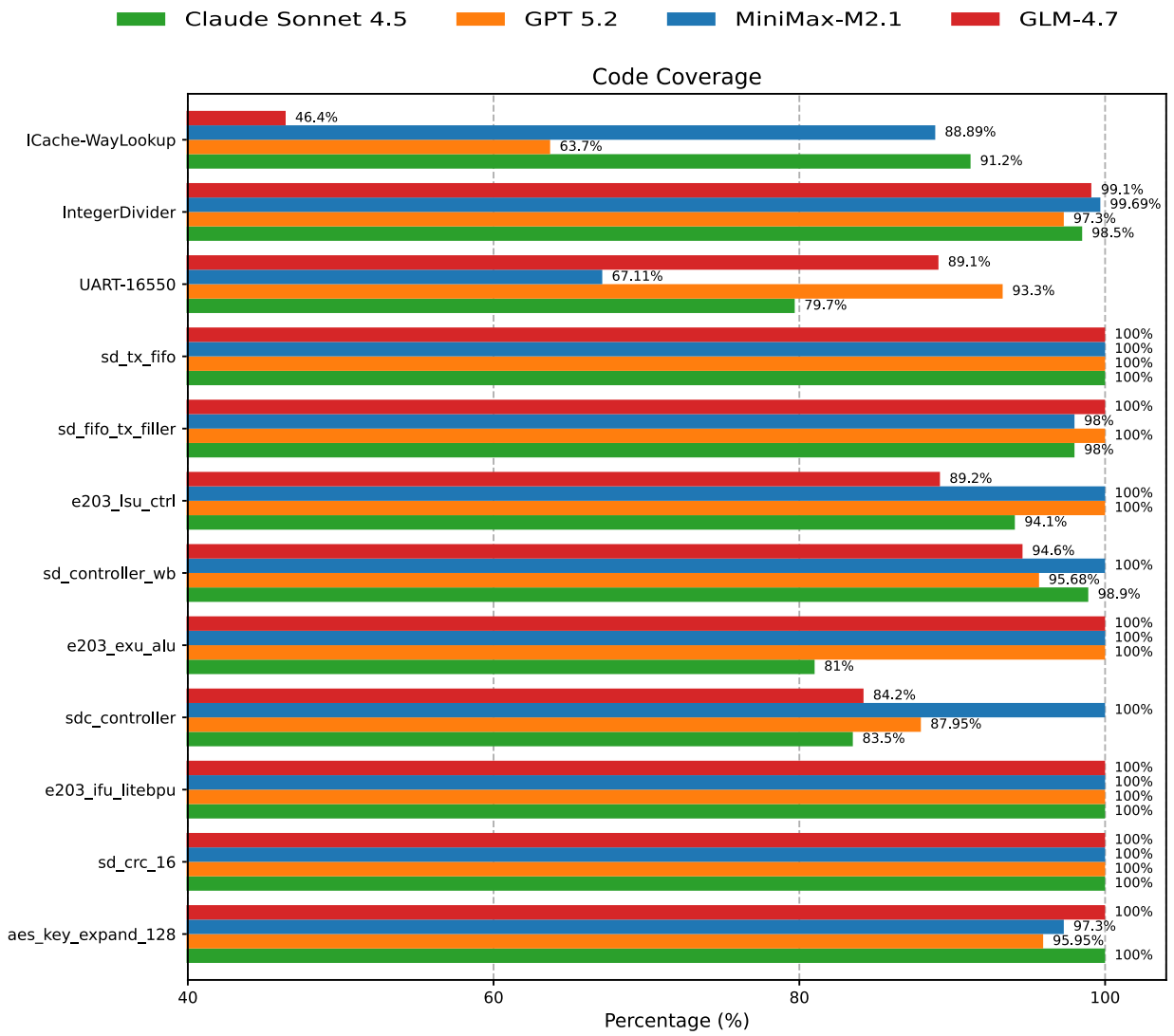
Benchmark Results: Code Coverage

LLMs	Coverage ≥ 95%
Claude Sonnet 4.5	7/12
GPT 5.2	8/12
MiniMax-M2.1	10/12
GLM-4.7	7/12

Comparison Across LLMs

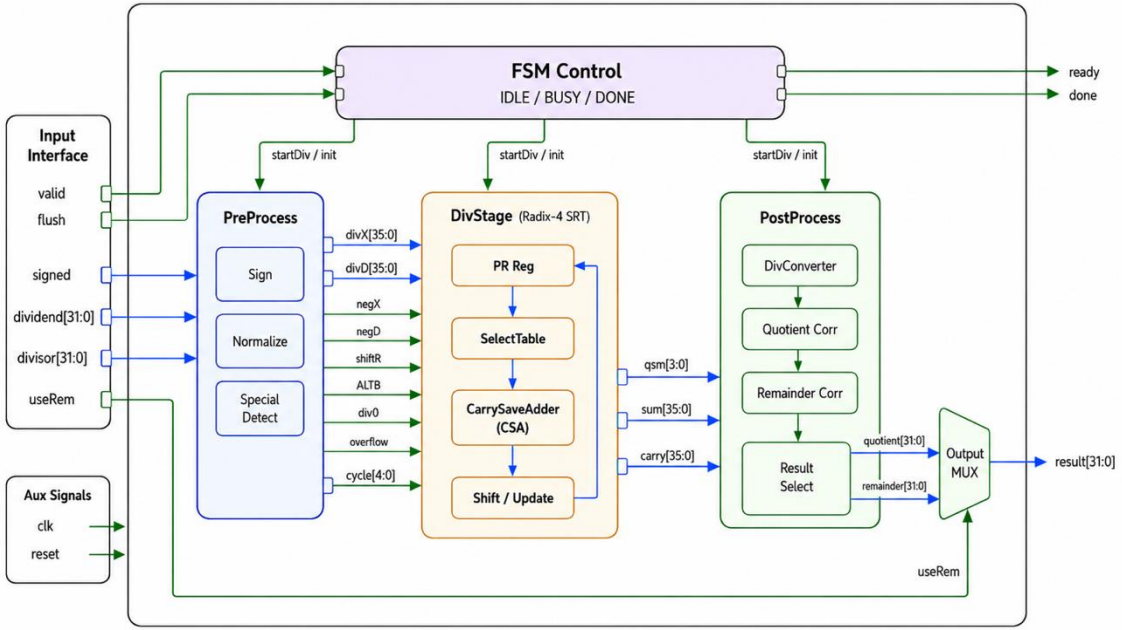
Module-Category	Coverage ≥ 95%
Control	7/16
Datapath	15/16
Protocol & Buffering	11/16

Comparison Across DUTs



Case 1: Higher Verification Efficiency

Radix-4 Integer Divider



1024 Verilog LOC

340 Scala LOC

11 Pins

Runtime Comparison

3 days manual effort → **45–51 min** UCAgent

~72x faster

Coverage Achieved

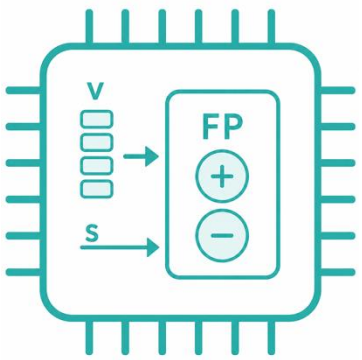
100% functional coverage

99.69% line coverage

Significantly reduces repetitive verification effort

Case 2: Finding Potential Bugs

Vector Floating-Point Mixed Adder



Chisel
751 LOC

Verilog
2980 LOC

125 pins

5 h
runtime

3
confirmed bugs

UT Bug Reproduction in XIANGSHAN

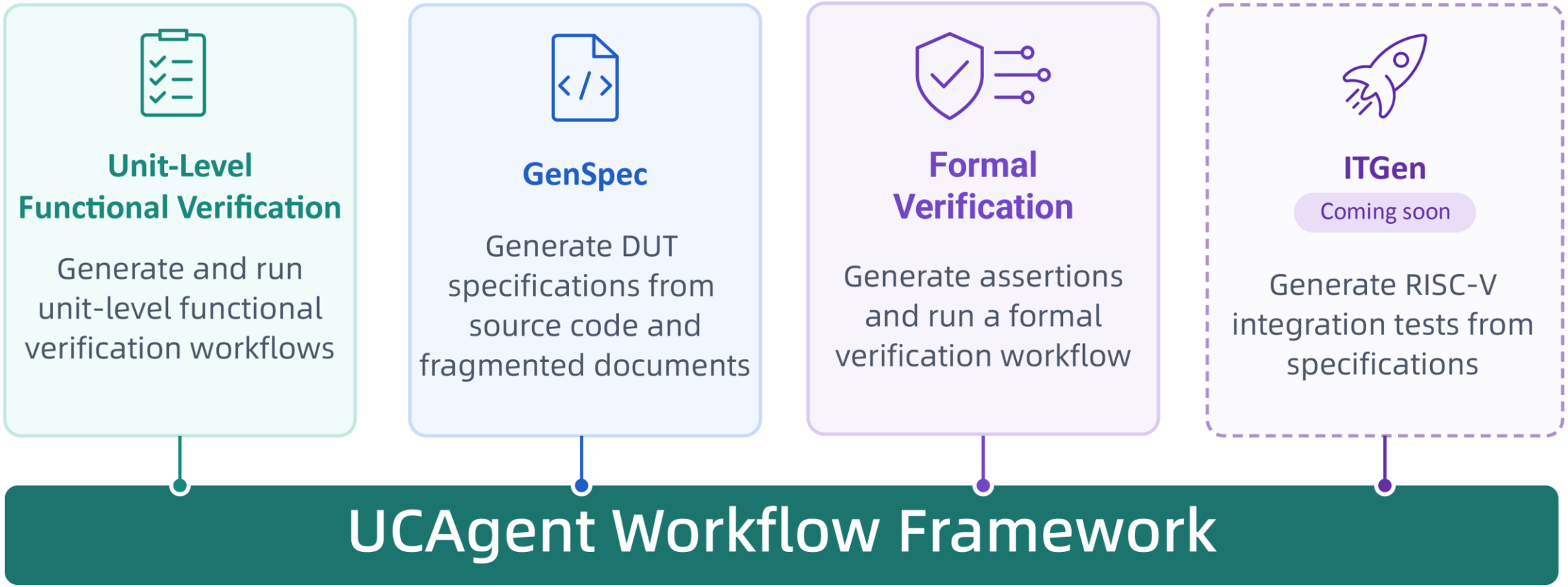
Average time < **4 hours**

PMP Checker 8 h / 1 bug	**Exu-Vsstatus** 4 h / 2 bugs
VecExceptionGen 2 h / 1 bug	**IFU Frontend** 1 h / 1 bug
ITTage 5 h / 2 bugs	**TageSC** 1 h / 1 bug

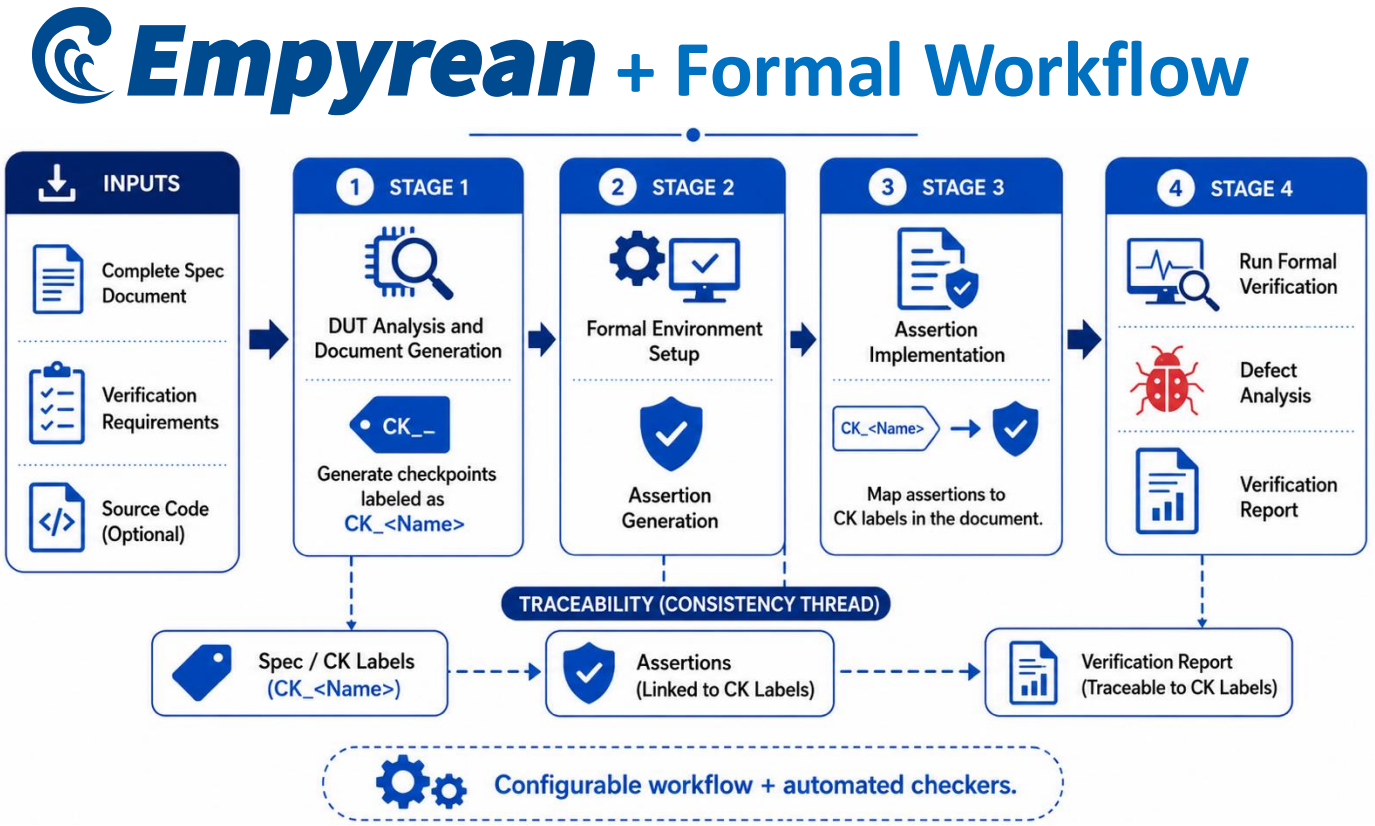
Helps engineers discover and reproduce real bugs

Current Capabilities

One framework foundation, multiple verification-related workflows



Current Capabilities: Formal Verification



Industrial Formal Integration

- Supports formal backends such as **Empyrean Formal MC** and **VC Formal**
- UCAgent generates structured assertions and CK labels
- Formal tools execute proof, analysis, and reporting

Example: ICache-Waylookup of Xiangshan

- 2k+ LOC
- 19 assertions and 5 covers

Before start, let's download the necessary packages

A. Access Web UI Only

1. **Connect Our WiFi**
   - SSID: BCC_26
   - Password: Bologna26
2. **Visit Address**
   - http://172.29.16.123:8800

B. Try UCAgent (Local)

1. **Pull Images from Github**

```
docker pull \
        ghcr.io/xs-mlvp/workshop:latest
```

See Next Page for execution commands

https://github.com/XS-MLVP/tutorial-records

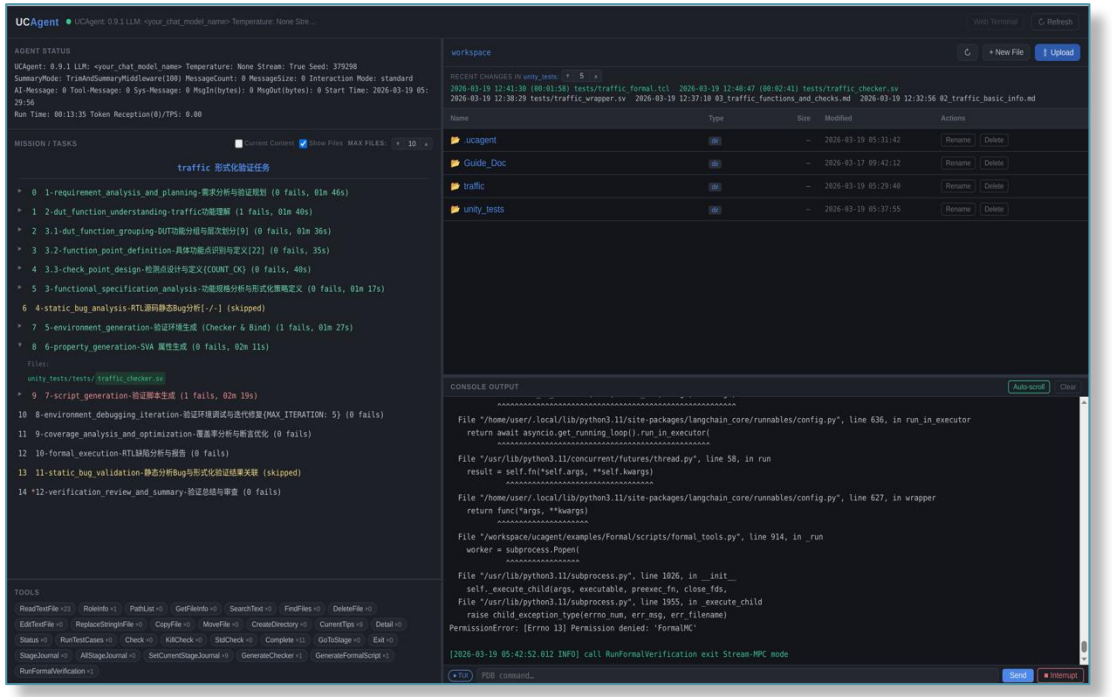
…/blob/master/workshop/README.md

Prerequisites

1. Start and run the image

```
# Local Computer with Shell
$ docker run -it \
           -p 8800:8800 \
           ghcr.io/xs-mlvp/workshop:latest
```

localhost:8800



2. Visit UCAgent WebUI

Open localhost:8800 in your browser

Welcome to Our Topic Table and Poster

Topic Table:

- Data: Lunch time on 10th & 11th
- Topic: Software-Driven Hardware Verification in the AI Era: From RTL-as-a-Package to AI Verification Agents

Poster Session:

- 10 th

Thanks for your time

Group

Website

Tutorial